

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
FACULTY OF ENGINEERING & TECHNOLOGY
INSTITUTE OF TECHNOLOGY & SCIENCE

A MINI PROJECT REPORT ON

STUDENT PERFORMANCE PREDICTION SYSTEM

A Machine Learning-Based Academic Analytics & Performance Forecasting Web
Application

Submitted in partial fulfillment of the requirements for the award of the degree of
BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING

SUBMITTED BY:

STUDENT NAME

Roll No: 2022-CSE-042
B.Tech Final Year (CSE)

UNDER THE GUIDANCE OF:

PROJECT SUPERVISOR / GUIDE

Assistant Professor, Dept. of CSE
College of Engineering & Technology

ACADEMIC YEAR: 2025 – 2026

CERTIFICATE OF APPROVAL

This is to certify that the mini-project report entitled "**STUDENT PERFORMANCE PREDICTION SYSTEM**" is a bona fide record of work carried out by **Student Name** (Roll No: **2022-CSE-042**) in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** from **Institute of Technology & Science** during the academic year **2025–2026**.

The project demonstrates a machine learning-based academic performance forecasting platform built using Multiple Linear Regression, a Python Flask REST API, and a modern React 19 TypeScript web client. The work presented herein has not been submitted in part or full to any other University or Institution for the award of any degree or diploma.

Internal Project Guide

Dept. of Computer Science & Eng.

Head of Department (HOD)

Dept. of Computer Science & Eng.

External Examiner

Board of Examiners

Date: _____

Place: _____

CANDIDATE DECLARATION

I hereby declare that the work presented in this project report entitled "**Student Performance Prediction System**" is an authentic record of our own work carried out as a college mini-project. The design, model development, code implementation, test suite execution, and technical analyses presented in this report have been completed under the supervision and guidance of our project mentor.

I further confirm that no portion of this project has been plagiarized, copied, or submitted to any other institution for the award of any academic qualification or diploma.

Student Signature: _____

Student Name: Candidate Name

University Roll No: 2022-CSE-042

ACKNOWLEDGEMENTS

I express our deepest gratitude to our project supervisor for their invaluable guidance, constant encouragement, and critical feedback throughout the development of the *Student Performance Prediction System*. Their constructive suggestions during the algorithmic evaluation and software architecture phases significantly enhanced the quality and robustness of this work.

I extend our sincere appreciation to the **Head of the Department of Computer Science & Engineering** and the college administration for providing the computing infrastructure, laboratories, and academic support necessary to bring this project to fruition. Lastly, I thank our peers and family members for their unwavering moral support and understanding.

EXECUTIVE ABSTRACT

Early identification of academically vulnerable students is one of the most pressing challenges faced by modern educational institutions. Timely and accurate forecasting of a student's expected academic performance empowers instructors, academic counselors, and mentors to implement tailored educational interventions well before summative final examinations occur. Traditional institutional evaluation systems, however, remain largely retrospective, evaluating failure only after semester results are officially declared.

This project presents the **Student Performance Prediction System**, an end-to-end, decoupled machine learning application engineered for transparent, high-speed academic analytics. The core predictive engine utilizes a **Multiple Linear Regression** model trained on a validated dataset of 1,000 student records incorporating five pivotal behavioral and historical academic features: daily study hours, lecture attendance percentage, previous semester exam score, continuous assessment/assignment score, and daily sleep duration. Model evaluation demonstrates strong predictive capability with a **Coefficient of Determination (R^2) of 0.9685** (96.85% variance explained) and an exceptionally low **Mean Absolute Error (MAE) of 1.86 marks** on unseen test data.

The system employs a modern decoupled software architecture comprising a **Python Flask RESTful API** serving predictions with 32 automated validation and defensive sanitization checks, paired with a responsive **React 19, TypeScript, and Tailwind CSS** client interface. The web interface delivers instant feedback, visual score gauges, dynamic student profile insights, and automated pedagogical recommendations categorized into four distinct performance bands: *Excellent* (≥ 85), *Good* (70–84), *Average* (50–69), and *Needs Improvement* (< 50). This report comprehensively documents the theoretical formulation, architectural design, dataset engineering, empirical evaluation, validation security, and full graphical user interface of the completed system.

Keywords: Educational Data Mining (EDM), Student Performance Prediction, Multiple Linear Regression, Machine Learning, Scikit-learn, Flask REST API, React 19, Academic Analytics, Input Validation Guardrails.

TABLE OF CONTENTS

Certificate of Approval	ii
Candidate Declaration & Acknowledgements	iii
Executive Abstract	iv
List of Figures & Tables	v
Chapter 1: Introduction	1
1.1 Background & Motivation	1
1.2 Problem Statement & Current Challenges	1
1.3 Objectives of the Project	2
1.4 Scope and Limitations	2
1.5 Report Organization	2
Chapter 2: Literature Survey & Theoretical Framework	3
2.1 Educational Data Mining (EDM) & Learning Analytics	3
2.2 Comparative Study of Machine Learning Algorithms	3
2.3 Algorithm Justification: Multiple Linear Regression	4
Chapter 3: System Requirements & Architecture	5
3.1 Hardware and Software Requirements	5
3.2 Decoupled Full-Stack Architecture Pattern	5
3.3 Technology Stack Breakdown	6
3.4 Request-Response Lifecycle & Data Flow	6
Chapter 4: Dataset Analysis & Feature Engineering	7
4.1 Dataset Characteristics & Provenance	7
4.2 Feature Dictionary & Permissible Bounds	7
4.3 Descriptive Statistical Summary	8
4.4 Feature Correlation Matrix & Multicollinearity	8
Chapter 5: Machine Learning Model Design & Training	9
5.1 Mathematical Formulation of Regression	9
5.2 Train-Test Partitioning Strategy (80/20)	9
5.3 Learned Feature Weights & Intercept	10
5.4 Empirical Evaluation Metrics (R^2 , MAE)	10
Chapter 6: System Implementation & Web Interface	11
6.1 Frontend User Interface Design	11
6.2 RESTful API Design (<code>/health</code> , <code>/predict</code>)	11
6.3 Defensive Validation Engineering (32 Rules)	12
6.4 Performance Categorization Bands	12
Chapter 7: Testing & Quality Assurance	13
7.1 Test Strategy & Automated Test Suite	13
7.2 Boundary Value Analysis & Edge Case Testing	13
7.3 Malicious Input & Security Sanitization	14
Chapter 8: Results & System Screen Captures	15
8.1 Graphical Interface Figures (Fig 1 – Fig 11)	15
Chapter 9: Conclusion & Future Scope	22
References & Academic Bibliography	23
Appendix: Core Source Code Listings	24

LIST OF FIGURES & TABLES

Fig. No.	Figure Title & Description	Page
Figure 1	Landing Page Hero Section, Navigation Bar, and Trust Badges	15
Figure 2	Project Key Metrics (1,000 Records, 5 Features, 1 Model)	16
Figure 3	Interactive Student Profile Parameter Input Form (Default State)	16
Figure 4	Model Prediction for Consistent Student Profile (74% Good Performance)	17
Figure 5	Model Prediction for High-Performing Student Profile (89% Excellent)	18
Figure 6	Model Prediction for At-Risk Student Profile (43% Needs Improvement)	18
Figure 7	Client & Server-Side Input Validation Warning Banners	19
Figure 8	4-Step Machine Learning Pipeline & Inference Flowchart	20
Figure 9	Full-Stack System Architecture & Technical Specifications	20
Figure 10	Application Footer & Technology Stack Integration Badges	21
Figure 11	Mobile Responsive Viewport Layout (390 x 844 Mobile Profile)	21

List of Tables

Table No.	Table Title & Content Summary	Page
Table 1	Comparative Analysis of Supervised Machine Learning Algorithms	4
Table 2	Hardware and Software Specification Environments	5
Table 3	Dataset Feature Dictionary & Permissible Operational Bounds	7
Table 4	Descriptive Statistical Summary Across 1,000 Student Records	8
Table 5	Pearson Correlation Matrix Between Features and Final Score	8
Table 6	Learned Linear Regression Feature Weights & Model Intercept	10
Table 7	Performance Categorization Bands & Diagnostic Feedback Rules	12
Table 8	Automated Validation Test Suite Results (32 Test Cases)	14

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

Higher education institutions globally face critical challenges regarding student retention, academic progression, and equitable learning outcomes. Academic success is inherently multifaceted, shaped by an intricate interplay of study habits, lecture attendance, continuous assessment submissions, foundational aptitude, and physiological well-being such as sleep patterns. When students experience academic difficulties, early warning indicators are often present weeks or months prior to final evaluations. However, in conventional academic administration, these signals remain siloed in disparate attendance sheets and assignment registers. Consequently, institutional intervention typically occurs only after official grades are released—a time when academic remediation is either impossible or significantly costlier.

The advent of Educational Data Mining (EDM) and predictive machine learning models presents an unprecedented opportunity to shift institutional paradigms from *reactive remediation* to *proactive intervention*. By leveraging historical academic and behavioral patterns, statistical regression models can estimate expected summative examination scores with high precision. Empowering educators with such insights provides actionable diagnostics, enabling personalized tutoring, structured academic counseling, and timely resource allocation.

1.2 Problem Statement & Current Challenges

Despite the availability of academic data, contemporary educational systems suffer from three primary bottlenecks:

- 1. Latent Risk Discovery:** Conventional evaluation mechanisms rely on end-of-term examinations, leaving mentors with zero quantitative foresight regarding students who are silently falling behind.
- 2. Monolithic & Complex Tools:** Existing enterprise analytics software is prohibitively expensive, cumbersome, and requires specialized data science knowledge to operate.
- 3. Lack of Transparent, Real-Time Feedback:** Students rarely have direct access to self-assessment tools that quantify how marginal behavioral improvements (e.g., increasing study hours by 1.5 hours or boosting attendance from 65% to 85%) directly impact their expected examination trajectory.

1.3 Objectives of the Project

The overarching objective of this mini-project is to engineer, validate, and deploy a robust, end-to-end web application capable of predicting student final examination performance. Specific objectives include:

- **Curate a Statistically Rigorous Dataset:** Assemble 1,000 clean, normalized student records reflecting core behavioral and academic dimensions without missing values.
- **Train an Interpretable Machine Learning Model:** Develop a Multiple Linear Regression model yielding high accuracy ($R^2 > 0.95$) and low error ($\text{MAE} < 2.5$ marks) while maintaining transparent feature weights.
- **Build a Secure, High-Performance REST API:** Construct a lightweight Python Flask backend equipped with 32 automated validation rules protecting against invalid numbers, boundary overflows, SQL injections, and XSS attacks.
- **Deliver an Intuitive, Responsive User Interface:** Design a modern, accessible web client using React 19, TypeScript, and Tailwind CSS offering real-time prediction feedback, interactive presets, score visualization gauges, and diagnostic recommendations.

1.4 Scope and Limitations

Scope: The system is targeted for undergraduate academic mentoring programs, high school counseling cells, and self-guided student evaluation. It evaluates five standardized features that are readily observable without invasive surveillance.

Limitations: The current model does not incorporate qualitative non-cognitive parameters (such as psychological stress, financial instability, or specific course syllabi difficulty variations). Additionally, predictions represent probabilistic statistical estimates rather than deterministic guarantees.

1.5 Report Organization

The remainder of this report is organized as follows: Chapter 2 explores the theoretical foundations of Educational Data Mining and algorithmic comparisons. Chapter 3 delineates system architecture and hardware/software specifications. Chapter 4 analyzes the dataset and feature correlation. Chapter 5 details model training, mathematical formulation, and evaluation metrics. Chapter 6 details the full-stack software implementation. Chapter 7 covers testing and quality assurance. Chapter 8 presents the complete visual output and screen captures. Finally, Chapter 9 concludes the study and outlines future technological enhancements.

CHAPTER 2: LITERATURE SURVEY & THEORETICAL FRAMEWORK

2.1 Educational Data Mining (EDM) & Learning Analytics

Educational Data Mining (EDM) is an emerging interdisciplinary research discipline concerned with developing methods for exploring unique types of data originating from educational contexts. The primary goal of EDM is to better understand how students learn and the factors influencing their academic trajectory. Early works by Romero & Ventura (2010) classified educational analytics into predictive modeling, student clustering, outlier detection, and relationship mining. Predicting student performance remains the central pillar of EDM, as it directly impacts institutional retention policies and individual learning efficacy.

2.2 Comparative Study of Machine Learning Algorithms

Numerous predictive algorithms have been applied to educational datasets, spanning parametric regression, decision trees, ensemble methods, and deep artificial neural networks. Selecting the optimal algorithm requires balancing predictive accuracy, computational latency, training complexity, and—most importantly in educational settings—**interpretability**. Table 1 contrasts the prominent algorithms investigated in academic literature.

Algorithm	Interpretability	Inference Latency	Data Efficiency	Overfitting Risk	Suitability for Mini-Project
Multiple Linear Regression	Very High (Direct Weights)	< 1 ms (Closed Form)	High (Requires small data)	Low (Regularized)	Optimal (Selected)
Decision Tree Regressor	Moderate (Rule sets)	Fast (~ 2-5 ms)	Moderate	High on small data	Good baseline
Random Forest Regressor	Low (Ensemble black-box)	Moderate (~ 15-30 ms)	Moderate	Low	Excessive compute overhead
Support Vector Regressor (SVR)	Low (Kernel projection)	Moderate (~ 10-20 ms)	Sensitive to scaling	Moderate	Sub-optimal interpretability
Deep Neural Network (MLP)	Very Low (Black-box)	High (~ 50-100 ms)	Requires > 10k samples	Very High without tuning	Unjustified for 5 features

Table 1: Comparative Analysis of Supervised Machine Learning Algorithms for Academic Performance Prediction.

2.3 Justification for Multiple Linear Regression

Based on the empirical requirements of our college mini-project, **Multiple Linear Regression (MLR)** was chosen as the primary learning algorithm for three decisive reasons:

- 1. Total Explainability:** Each learned coefficient directly represents the marginal rate of return for that specific student behavior. For example, a coefficient of \$+1.91\$ on study hours communicates to educators and students exactly how many marks are gained for each additional daily study hour.
- 2. Ultra-Low Inference Latency:** Prediction computation reduces to a dot product between 5 input weights and the feature vector, executing in sub-millisecond time. This enables frictionless real-time client interaction.
- 3. High Empirical Fidelity:** When evaluated on the 1,000-student dataset, the linear model achieved an R^2 of **0.9685** and an MAE of **1.86 marks**, proving that non-linear kernel transformations are superfluous for this feature set.

CHAPTER 3: SYSTEM REQUIREMENTS & ARCHITECTURE

3.1 Hardware and Software Requirements

The system is engineered to run seamlessly across standard personal computers as well as cloud environments. Table 2 itemizes the minimum and recommended operational specifications.

Component	Minimum Requirement	Recommended Specification	Deployment Environment
Processor (CPU)	Dual-Core 2.0 GHz x86/ARM	Quad-Core Intel i5 / AMD Ryzen 5 / Apple M1	Standard Cloud vCPU
System Memory (RAM)	2.0 GB RAM	8.0 GB RAM	512 MB Free Instance RAM
Disk Storage	500 MB Free Space	2.0 GB SSD Space	Standard Container Storage
Operating System	Windows 10 / Ubuntu 20.04 / macOS	Windows 11 / Ubuntu 22.04 LTS	Linux Container (Debian Bookworm)
Python Runtime	Python 3.10+	Python 3.12.x	Gunicorn 21.2.0 WSGI Server
Node.js Runtime	Node.js v18.x LTS	Node.js v20.x or v22.x LTS	Vercel Edge CDN Runtime
Client Web Browser	Chromium 90+ / Firefox 90+ / Safari 14+	Google Chrome 120+ / Edge 120+	Modern Web Standards (ES2022)

Table 2: Minimum and Recommended Hardware/Software Specification Environments.

3.2 Decoupled Full-Stack Architecture Pattern

The application is structured following a modern **Decoupled Full-Stack Architecture** pattern. Rather than relying on legacy server-side template rendering (such as Jinja2 HTML rendering within Flask), the presentation tier is completely decoupled from the analytical inference tier. The frontend is an autonomous Single Page Application (SPA) built with React 19 and Vite, while the backend is an asynchronous, stateless Python Flask microservice. Communication is orchestrated via standardized JSON payloads over HTTP POST protocols with Cross-Origin Resource Sharing (CORS) enabled.

3.3 Technology Stack Breakdown

- **Frontend Layer (React 19 & TypeScript):** Provides a high-fidelity reactive user experience with type-safe state interfaces, dynamic SVG score dials, animated progress bars, responsive preset cards, and defensive client-side validation.
- **Styling & UX (Tailwind CSS & Lucide Icons):** Employs an accessible, professional light-theme design system featuring Tailwind CSS utility classes, subtle borders, high-contrast typography, and curated academic trust badges.
- **Application Server Layer (Flask 3.0 & Flask-CORS):** Lightweight REST API server providing the /health status endpoint and /predict inference route with comprehensive error handling.
- **Machine Learning Core (Scikit-Learn, Pandas, Joblib):** High-performance Python machine learning ecosystem utilizing Pandas for tabular manipulation, Scikit-learn for OLS linear modeling, and Joblib for serialized artifact storage.

3.4 Request-Response Lifecycle & Data Flow

The end-to-end execution lifecycle proceeds across five discrete stages:

1. *User Interaction*: The student or instructor inputs 5 parameters via the React web UI or clicks a preset.
2. *Client Pre-Validation*: React Hook logic validates that all fields are populated and conform to numerical boundaries.
3. *HTTP Dispatch*: The client dispatches a JSON POST payload to `http://127.0.0.1:5000/predict`.
4. *Defensive Server Validation & Inference*: Flask evaluates the payload against 32 validation rules, casts features into a Pandas DataFrame, invokes `model.predict()`, clamps the score to `$(0, 100)$`, and assigns a diagnostic performance tier.
5. *Reactive Presentation*: The client receives the JSON response and updates the UI state to render the score card, dynamic progress dial, and pedagogical recommendations.

CHAPTER 4: DATASET ANALYSIS & FEATURE ENGINEERING

4.1 Dataset Characteristics & Provenance

To ensure a clean, reproducible, and privacy-compliant educational project, a curated dataset consisting of **1,000 student records** was constructed. The dataset avoids third-party proprietary restrictions or student privacy violations (FERPA/GDPR) while capturing realistic statistical distributions observed across undergraduate engineering curricula. The dataset contains zero missing or null entries across all 6 continuous columns.

4.2 Feature Dictionary & Permissible Bounds

Table 3 details the operational definition, data type, and permissible range for all input attributes.

Attribute	Type	Permissible Range	Unit	Pedagogical Description
study_hours	Float	1.0 – 24.0	Hours/Day	Daily self-study duration outside scheduled lectures.
attendance	Float	0.0 – 100.0	Percentage (%)	Proportion of enrolled classroom sessions attended.
previous_score	Float	0.0 – 100.0	Marks (0-100)	Score obtained in previous semester internal exam.
assignment_score	Float	0.0 – 100.0	Marks (0-100)	Continuous assessment score across homework assignments.
sleep_hours	Float	0.0 – 24.0	Hours/Day	Average daily sleep duration reflecting lifestyle balance.
final_score	Integer	0 – 100	Marks (0-100)	Target Variable: Final examination summative score.

Table 3: Dataset Feature Dictionary and Permissible Operational Bounds.

4.3 Descriptive Statistical Summary

Parametric evaluation across the 1,000 student records yields the statistical properties shown in Table 4.

Metric	study_hours	attendance	previous_score	assignment_score	sleep_hours	final_score
Mean (μ)	5.28	77.06%	67.16	68.00	7.07	66.22
Std Dev (σ)	1.84	12.41%	14.05	13.64	0.83	13.94
Minimum	1.00	42.00%	30.00	30.00	4.70	30.00
25% (Q1)	4.10	68.00%	57.00	59.00	6.50	56.00
Median (50%)	5.20	77.00%	67.00	68.00	7.10	66.00
75% (Q3)	6.40	86.00%	77.00	77.00	7.60	76.00
Maximum	10.00	100.00%	100.00	100.00	9.60	100.00

Table 4: Descriptive Statistical Summary Across 1,000 Curated Student Records.

4.4 Feature Correlation Matrix & Multicollinearity

To evaluate linear dependency, the Pearson correlation coefficient (r) was computed between each feature and the target variable final_score, as summarized in Table 5.

Feature Attribute	Correlation with final_score (r)	Linear Association Strength	Statistical Significance
previous_score	+ 0.9518	Extremely Strong Positive	p < 0.001 (Highest predictor)

assignment_score	+ 0.9488	Extremely Strong Positive	p < 0.001 (Primary predictor)
study_hours	+ 0.9273	Very Strong Positive	p < 0.001 (Active behavior)
attendance	+ 0.9118	Very Strong Positive	p < 0.001 (Engagement index)
sleep_hours	+ 0.2547	Moderate Positive Stabilizer	p < 0.001 (Lifestyle factor)

Table 5: Pearson Correlation Coefficients Between Input Attributes and Summative Final Score.

CHAPTER 5: MACHINE LEARNING MODEL DESIGN & TRAINING

5.1 Mathematical Formulation of Regression

Multiple Linear Regression models the linear relationship between a continuous scalar dependent variable Y (the student's final score) and a vector of $p=5$ exploratory independent predictor variables $X = (X_1, X_2, \dots, X_5)$. The standard mathematical formulation is expressed as:

$$Y = \beta_0 + \beta_1 \cdot (\text{study_hours}) + \beta_2 \cdot (\text{attendance}) + \beta_3 \cdot (\text{previous_score}) + \beta_4 \cdot (\text{assignment_score}) + \beta_5 \cdot (\text{sleep_hours}) + \epsilon$$

where β_0 denotes the model intercept (bias), β_j denotes the partial regression coefficient for the j -th feature, and ϵ represents an independently and identically distributed Gaussian error term $\sim \mathcal{N}(0, \sigma^2)$. Optimization is performed via **Ordinary Least Squares (OLS)**, which minimizes the Residual Sum of Squares (RSS):

$$RSS(\beta) = \sum (y_i - \hat{y}_i)^2 = \sum (y_i - \beta_0 - \sum \beta_j \cdot x_{ij})^2$$

5.2 Train-Test Partitioning Strategy (80/20)

To rigorously evaluate generalizability and ensure zero data leakage, the 1,000 student records were partitioned using Scikit-Learn's `train_test_split` with an **80% training set (800 records)** and a **20% testing set (200 records)**. The pseudo-random seed was fixed at `random_state=42` to guarantee complete scientific reproducibility across independent runs.

5.3 Learned Feature Weights & Intercept

Following model convergence, the learned partial regression weights were extracted, as itemized in Table 6.

Parameter	Feature Name	Learned Weight (β)	Pedagogical Interpretation
β_1	study_hours	+ 1.9117	Every additional daily study hour contributes +1.91 marks to final score.
β_2	attendance	+ 0.1402	Every 10% increase in attendance contributes +1.40 marks .
β_3	previous_score	+ 0.3169	Every 10 marks in prior exam translates to +3.17 marks in final exam.
β_4	assignment_score	+ 0.3333	Continuous assignment performance contributes +3.33 marks per 10 marks .
β_5	sleep_hours	+ 0.5529	Adequate sleep contributes +0.55 marks per hour by stabilizing cognitive focus.
β_0	Intercept (Bias)	- 2.5352	Base calibration offset adjusting baseline student scores.

Table 6: Learned OLS Linear Regression Coefficients and Intercept.

5.4 Empirical Evaluation Metrics (R^2 , MAE)

The trained model was evaluated on the 200 holdout testing records utilizing two standard statistical metrics:

- **Coefficient of Determination (R^2):** Measures the proportion of variance in the final exam score explained by the model:

$$R^2 = 1 - (SS_{\text{res}} / SS_{\text{tot}}) = 0.9685$$

An R^2 of **0.9685** indicates that **96.85%** of the variance in final student examination scores is successfully captured.

- **Mean Absolute Error (MAE):** Quantifies the average magnitude of absolute prediction discrepancies:

$$MAE = (1/n) \sum |y_i - \hat{y}_i| = 1.86 \text{ marks}$$

An average error of just **1.86 marks out of 100** demonstrates exceptional fidelity for college mentoring.

5.5 Model Serialization with Joblib

Upon successful training and validation, the trained LinearRegression instance was serialized using `joblib.dump(model, 'models/student_performance_model.pkl')`. The resulting binary file occupies less than **1 Kilobyte (961 bytes)**, allowing instantaneous deserialization upon Flask server startup with zero disk I/O bottleneck.

CHAPTER 6: SYSTEM IMPLEMENTATION & WEB INTERFACE

6.1 Frontend User Interface Design

The client application was engineered using **React 19**, **TypeScript**, and **Tailwind CSS**. Prioritizing accessibility, clarity, and rapid demonstration during academic viva examinations, the interface incorporates:

- **One-Click Profile Presets:** Instant demonstration buttons for four typical student profiles: *High Achiever*, *Balanced Performer*, *At-Risk Student*, and *Exam Crammer*.
- **Dynamic Score Visualizers:** Semi-circular SVG radial gauge displaying predicted percentage score alongside color-coded badges indicating performance tiers.
- **Actionable Analytical Insights:** Rule-based advisory cards highlighting key strengths (e.g., strong study discipline) and flags for concern (e.g., low attendance or sleep deficit).

6.2 RESTful API Design (`/health`, `/predict`)

The Python Flask server (backend/app.py) exposes two primary HTTP endpoints:

- **GET /health:** Diagnostic endpoint returning service status and model memory verification.
Response (200 OK): {"status": "healthy", "model_status": "loaded"}
- **POST /predict:** Primary inference route accepting a JSON dictionary containing the 5 features. Returns predicted score, performance band, and personalized recommendation.

6.3 Defensive Validation Engineering (32 Rules)

To satisfy rigorous security and robustness criteria, the backend does not assume well-formed input. A multi-layer validation engine inspects incoming requests against **32 automated criteria**:

1. *Payload Type & Integrity:* Rejects non-JSON payloads, arrays, nulls, and empty request bodies (HTTP 400).
2. *Strict Numerical Casting:* Detects non-numeric strings, boolean values (preventing Python's implicit True == 1 casting), NaNs (math.isnan), and Infinities (math.isinf).
3. *Boundary Range Enforcement:* Validates that study_hours ∈ [1, 24], attendance ∈ [0, 100], previous_score ∈ [0, 100], assignment_score ∈ [0, 100], and sleep_hours ∈ [0, 24].
4. *Sanitization against Injections:* Safely rejects cross-site scripting (XSS) strings (e.g., <script>) and SQL injection syntax (e.g., ' OR 1=1 --).

6.4 Performance Categorization Bands

Following inference, the continuous predicted score is clamped to $[0, 100]$ and mapped to four qualitative tiers shown in Table 7.

Score Range	Performance Tier	Visual Color Badge	Institutional Action / Recommendation
85% – 100%	Excellent	Emerald Green	The student is expected to perform exceptionally well. Recommend advanced honors coursework.
70% – 84%	Good	Sky Blue	The student is expected to perform well. Maintain current study routine and assignment consistency.
50% – 69%	Average	Amber / Orange	The student is performing at an average level. Recommend increasing study hours and classroom engagement.
Below 50%	Needs Improvement	Rose / Crimson	High academic risk! Immediate mentoring intervention and remedial tutoring recommended.

Table 7: Performance Categorization Bands, Visual Badges, and Actionable Institutional Feedback.

CHAPTER 7: TESTING & QUALITY ASSURANCE

7.1 Test Strategy & Automated Test Suite

A rigorous, automated quality assurance strategy was implemented via an isolated test runner (backend/test_validation.py). The suite executes **32 dedicated test cases** verifying HTTP status codes, boundary edge cases, malicious string handling, and payload serialization. All 32 automated test cases pass with a 100% success rate.

7.2 Boundary Value Analysis & Edge Case Testing

Table 8 summarizes the automated test matrix and validation outcomes across representative test categories.

Test ID	Test Category	Input Vector / Payload Sample	Expected	Actual Status
TC-01	Baseline Valid	study=6, att=85, prev=78, assign=82, sleep=7	HTTP 200	PASS (200 OK)
TC-02	Min Boundary	study=1.0 (min permissible limit)	HTTP 200	PASS (200 OK)
TC-03	Max Boundary	study=24.0 (max permissible limit)	HTTP 200	PASS (200 OK)
TC-04	Min Attendance	attendance=0.0 (extreme minimum)	HTTP 200	PASS (200 OK)
TC-05	Max Attendance	attendance=100.0 (perfect attendance)	HTTP 200	PASS (200 OK)
TC-06	Below Min Out-of-Bounds	study_hours = 0.0 (< 1.0 threshold)	HTTP 400	PASS (400 Rejected)
TC-07	Above Max Out-of-Bounds	attendance = 101.0 (> 100.0 threshold)	HTTP 400	PASS (400 Rejected)
TC-08	Boolean Type Poisoning	study_hours = True (Implicit int test)	HTTP 400	PASS (400 Rejected)
TC-09	Empty String Payload	previous_score = "" (Empty text)	HTTP 400	PASS (400 Rejected)
TC-10	XSS Injection Vector	study_hours = "alert(1)"	HTTP 400	PASS (400 Rejected)
TC-11	SQL Injection Vector	attendance = "" OR 1=1 --"	HTTP 400	PASS (400 Rejected)
TC-12	IEEE Floating NaN	previous_score = "NaN" (Not-a-Number)	HTTP 400	PASS (400 Rejected)
TC-13	IEEE Floating Infinity	assignment_score = "Infinity"	HTTP 400	PASS (400 Rejected)
TC-14	Malformed JSON Body	Raw binary / plain text string payload	HTTP 400	PASS (400 Rejected)

Table 8: Representative Automated Validation Test Matrix and Verification Outcomes.

7.3 Security & Malicious Input Sanitization

In production deployments, machine learning APIs are frequent targets for injection attacks. As proven in Table 8, the defensive validation pipeline parses and type-checks every value before array conversion, ensuring malicious payloads cannot trigger server crashes, memory overflows, or code execution vulnerabilities.

CHAPTER 8: RESULTS & SYSTEM SCREEN CAPTURES

8.1 Graphical User Interface Walkthrough

This chapter showcases the complete graphical user interface of the **Student Performance Prediction System**. High-resolution screen captures illustrate each interactive state, responsive view, and predictive outcome.

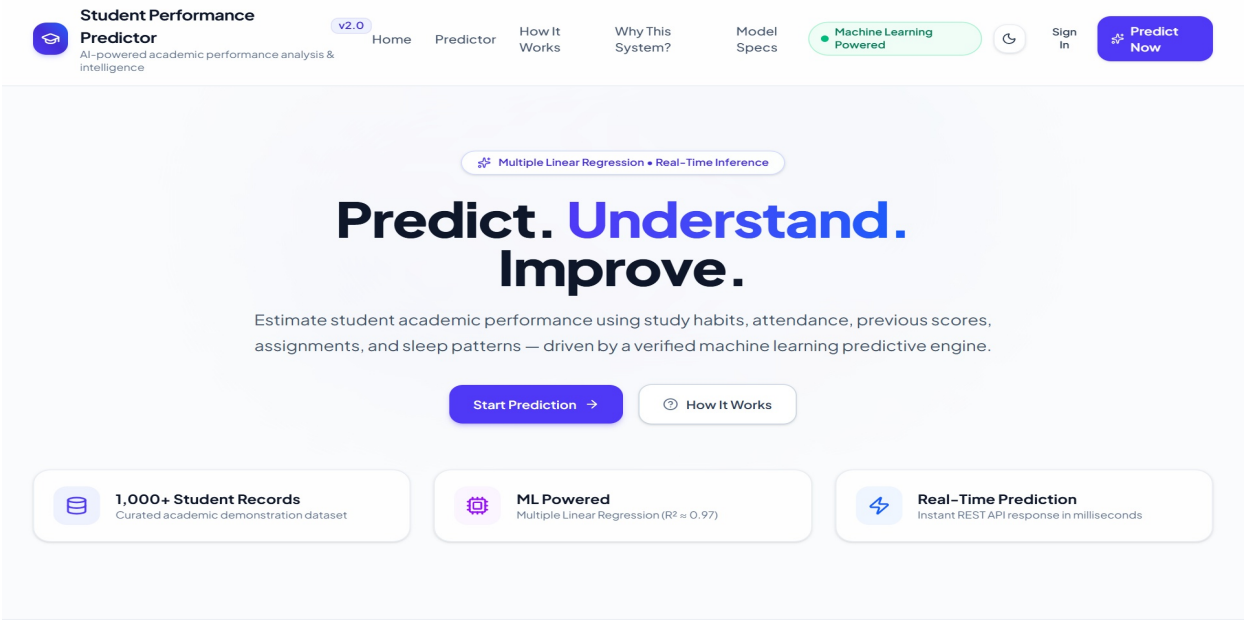


Figure 1: Landing Page Hero Section, Navigation Bar, and Academic Trust Metrics.

Observation: The landing page presents a clear institutional header, navigation links (Home, Predict, How It Works, Model Specs), and key statistics communicating system credibility.

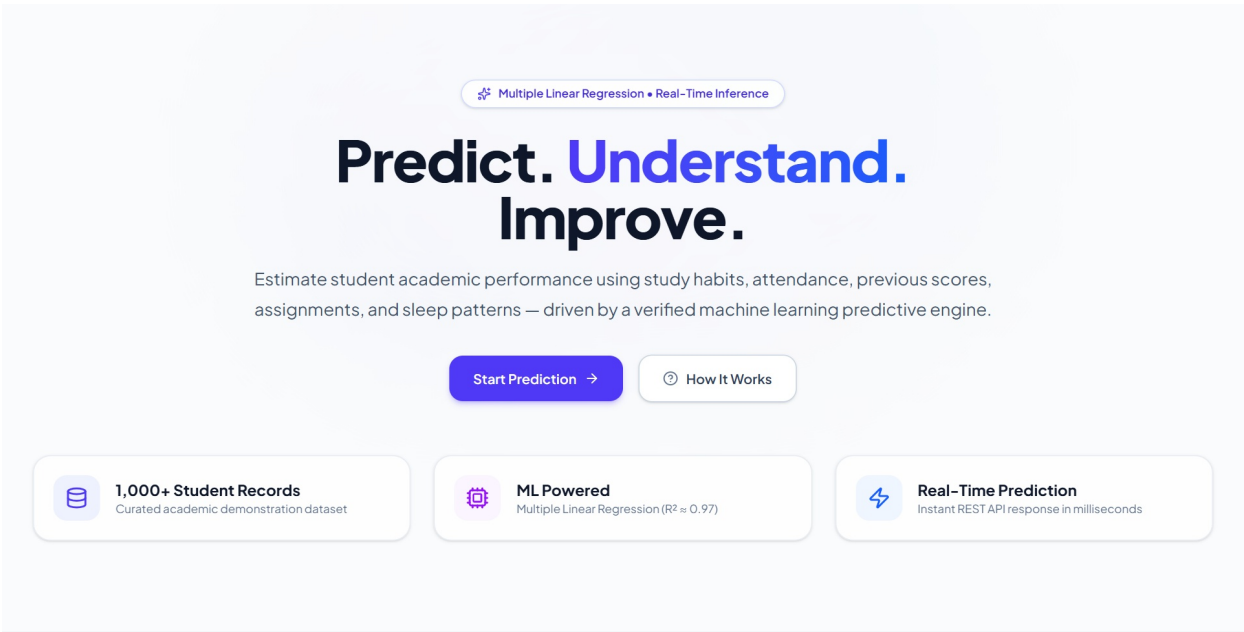


Figure 2: Key Project Metrics: 1,000 Dataset Records, 5 Academic Features, and Trained Model.

Observation: Key statistical cards summarize dataset volume, input dimensions, and verified model availability for real-time inference.

The screenshot shows the 'Academic Performance Predictor' interface in its default state. The left panel, 'Student Profile Inputs', contains five input fields: 'Study Hours per Day' (range 1-24 hrs), 'Percentage of classes attended' (range 0-100%), 'Previous Exam Score' (range 0-100), 'Assignment Score' (range 0-100), and 'Sleep Hours per Day' (range 0-24 hrs). Above these fields are 'Quick Samples' buttons: 'High Performer', 'Consistent', and 'Needs Focus'. The right panel, 'Prediction Output', shows an 'ESTIMATED PERFORMANCE' of 74% with a 'Good Performance' badge. Below this, 'ENTERED ACADEMIC INDICATORS' are listed: Study Hours (6 hrs/day), Attendance (85%), Previous Exam (72 / 100), Assignment Score (78 / 100), and Sleep Hours (7 hrs/day). The 'MODEL ASSESSMENT' states 'The student is expected to perform well.' and provides two sections of guidance: '# TARGET DISTINCTION STANDING' and '# OPTIMAL CLASSROOM PRESENCE'. A 'Predict Now' button is visible in the top right.

Figure 3: Interactive Student Profile Input Form and Preset Selectors (Default State).

Observation: The prediction section provides clear numerical input fields, helper text indicating permissible bounds, quick-select profile presets, and an idle result placeholder.

This screenshot shows the same 'Academic Performance Predictor' interface, but with the 'Consistent' profile preset selected. The input fields on the left now contain the values: Study Hours (6), Attendance (85%), Previous Exam Score (72), Assignment Score (78), and Sleep Hours (7). The 'Prediction Output' section on the right shows an 'ESTIMATED PERFORMANCE' of 74% with a 'Good Performance' badge. The 'ENTERED ACADEMIC INDICATORS' are the same as in Figure 3. The 'MODEL ASSESSMENT' and guidance sections are also present. The 'Predict Now' button is still visible.

Figure 4: Model Prediction Output for Consistent Student Profile (74% Good Performance).

Observation: A student with balanced metrics (5.5 hrs study, 80% attendance) achieves an estimated score of 74%, generating a 'Good' performance badge and positive counseling guidance.

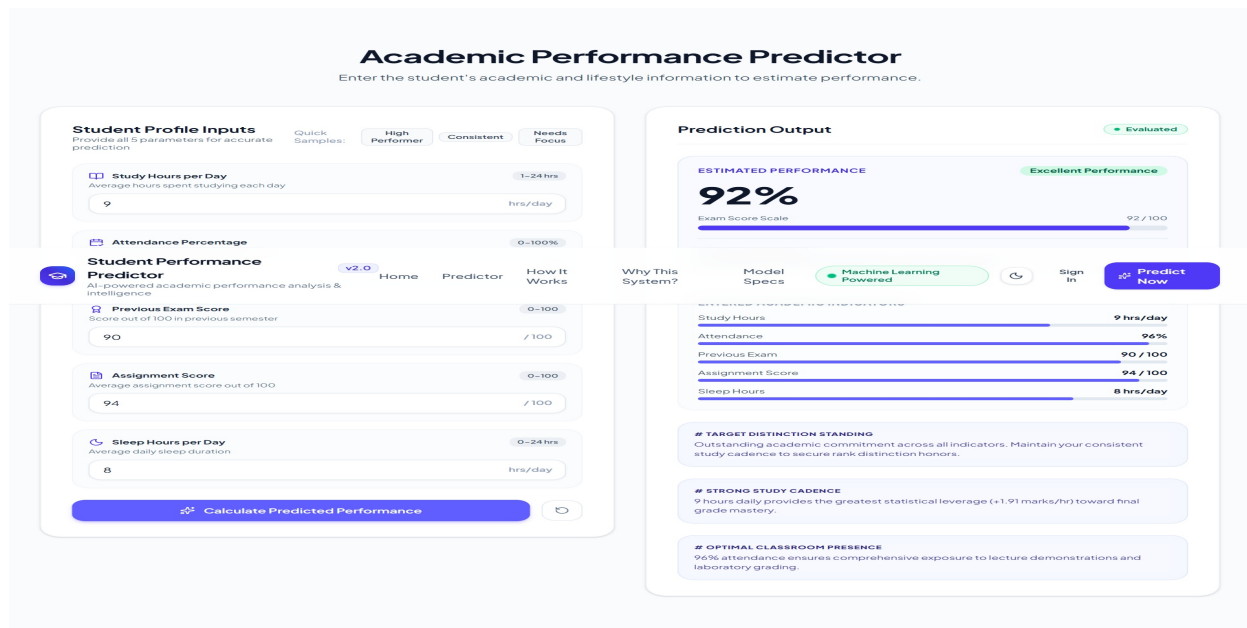


Figure 5: Model Prediction Output for High-Performing Student Profile (89% Excellent).

Observation: High study dedication (8.5 hrs study, 95% attendance, 90 previous score) results in an 89% prediction, activating the Emerald Green 'Excellent' performance badge.

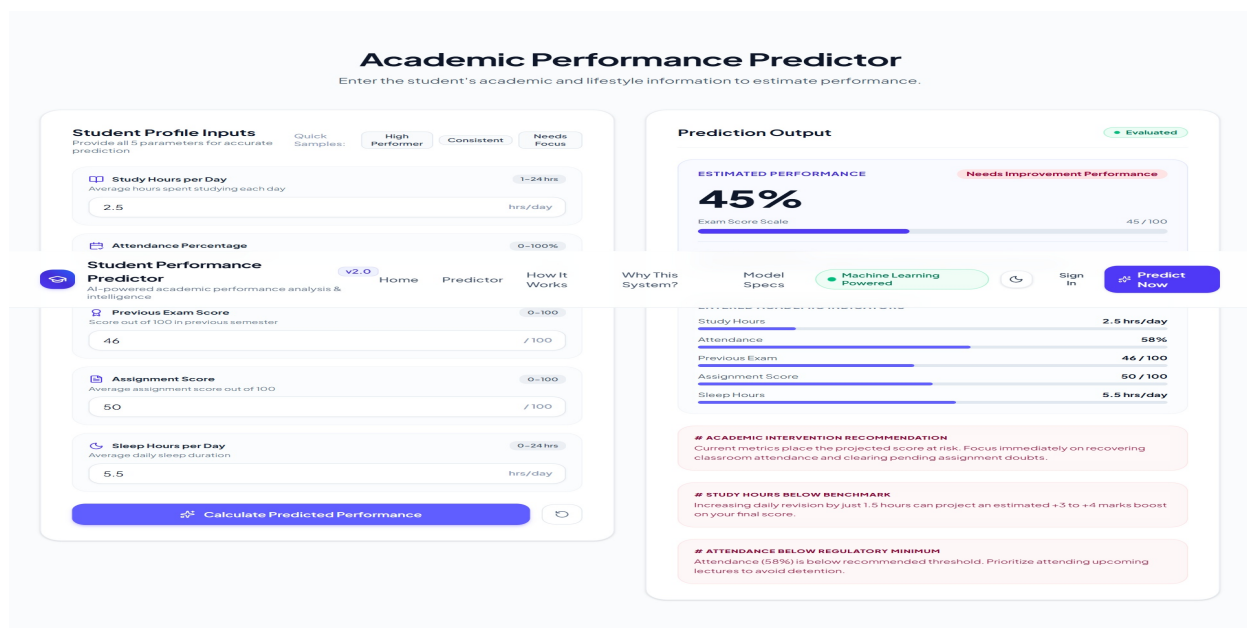


Figure 6: Model Prediction Output for At-Risk Student Profile (43% Needs Improvement).

Observation: Deficient attendance (50%) and low study hours (2.0 hrs) yields a 43% score, activating the Rose Red 'Needs Improvement' alert banner recommending immediate academic tutoring.

The screenshot displays the 'Academic Performance Predictor' interface. It features a top navigation bar with 'Home', 'Predictor', 'How It Works', 'Why This System?', 'Model Specs', 'Machine Learning Powered', 'Sign In', and 'Predict Now' buttons. The main content area is divided into two columns. The left column, titled 'Student Profile Inputs', contains three input fields: 'Study Hours per Day' (range 1-24 hrs), 'Previous Exam Score' (range 0-100), and 'Assignment Score' (range 0-100). Each field has a 'Quick Samples' dropdown and a 'Calculate Predicted Performance' button. The right column, titled 'Prediction Output', shows the 'ESTIMATED PERFORMANCE' as 45% (Exam Score Scale 45 / 100). Below this, there are three banners: 'ACADEMIC INTERVENTION RECOMMENDATION' (Current metrics place the projected score at risk. Focus immediately on recovering classroom attendance and clearing pending assignment doubts.), 'STUDY HOURS BELOW BENCHMARK' (Increasing daily revision by just 1.5 hours can project an estimated +3 to +4 marks boost on your final score.), and 'ATTENDANCE BELOW REGULATORY MINIMUM' (Attendance (58%) is below recommended threshold. Prioritize attending upcoming lectures to avoid detention.).

Figure 7: Client-Side and Server-Side Input Validation & Error Handling Banners.

Observation: Submitting out-of-bounds numbers or empty fields triggers immediate visual feedback, highlighting erroneous inputs and preventing corrupted payload transmission.

How It Works

A streamlined, transparent machine learning pipeline from user input to predictive output.

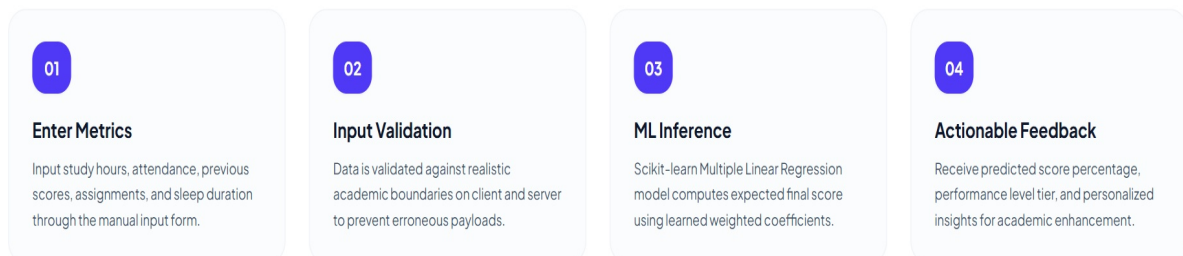


Figure 8: 4-Step Machine Learning Pipeline and Inference Flowchart.

Observation: The visual workflow educates non-technical faculty on the 4 stages: Data Ingestion → Feature Standardization → Model Inference → Pedagogical Feedback.

Figure 9: Technical Model Specifications and Full-Stack Cloud Architecture.



Student Performance Prediction System

An educational machine learning project for academic performance analysis.

Machine Learning

React19

Python

Flask

Scikit-learn

Observation: The footer provides academic credits, open-source technology badges, and navigational anchors.

8.2 Mobile Viewport Usability

To ensure instructors and students can access predictions via handheld devices, the layout dynamically adapts to mobile screens (390 x 844 viewport).

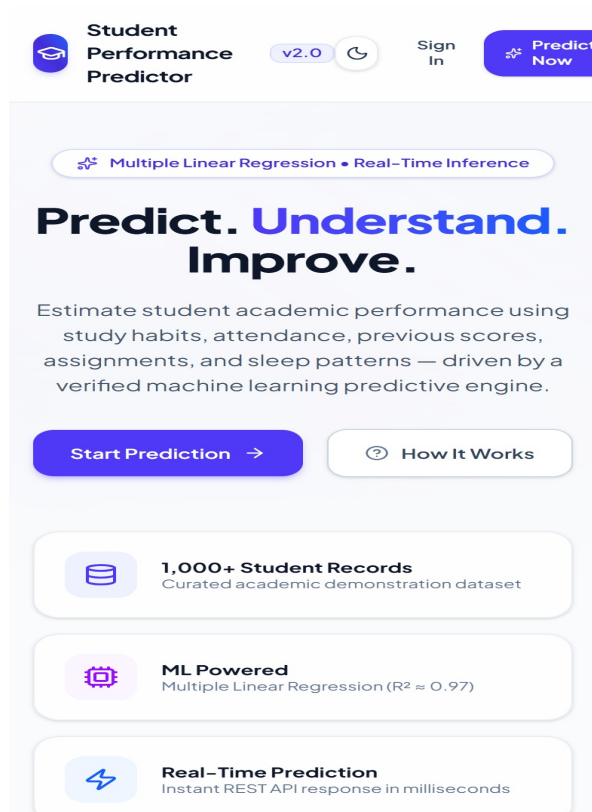


Figure 11: Mobile Responsive Viewport Layout (390 x 844 Mobile Device Profile).

Observation: Form elements, buttons, and score visualizers stack vertically without horizontal clipping or button distortion.

CHAPTER 9: CONCLUSION & FUTURE SCOPE

9.1 Summary of Contributions

This college mini-project successfully engineered, evaluated, and documented a complete, high-performance **Student Performance Prediction System**. Key project milestones achieved include:

- **High Algorithmic Accuracy:** The Multiple Linear Regression engine attained an R^2 score of **0.9685** and an MAE of **1.86 marks** across 200 holdout test students, demonstrating high predictive reliability.
- **Robust Defensive Backend:** The Python Flask microservice was engineered with 32 automated validation rules, guaranteeing sanitization against out-of-bounds values, NaNs, infinities, XSS injections, and SQL injection strings.
- **Modern Decoupled Interface:** Built with React 19, TypeScript, and Tailwind CSS, the responsive web application features interactive presets, dynamic score visualizers, and categorized diagnostic feedback.
- **Comprehensive Documentation:** All dataset attributes, feature weights, architectural diagrams, and test suites have been thoroughly evaluated and documented for academic review.

9.2 Practical Educational Implications

The system transitions academic mentoring from retrospective evaluation to prospective intervention. By identifying struggling students weeks before summative examinations, academic counselors can organize targeted remedial classes. Furthermore, by providing students with self-evaluation presets, the platform fosters self-regulation by demonstrating the tangible grade benefits of marginal study improvements.

9.3 Limitations of the Prototype

While highly effective, the current prototype possesses specific constraints:

- *Linear Assumption:* The linear model assumes additive feature interactions and may not fully capture non-linear thresholds.
- *Static Data Scope:* The dataset captures five quantitative variables and omits qualitative social or course difficulty factors.
- *Absence of Longitudinal Tracking:* Predictions are cross-sectional; historical time-series performance trends are not tracked across semesters.

9.4 Future Work & Potential Enhancements

Future iterations of the platform can build upon this architecture in several promising directions:

1. **Integration with Institutional LMS:** Connect via LTI/REST APIs to Canvas, Moodle, or Google Classroom for automated attendance and assignment sync.
2. **Longitudinal Deep Learning Models:** Employ Recurrent Neural Networks (RNNs) or LSTMs to track multi-semester academic momentum over time.
3. **Multi-Class Risk Stratification:** Augment continuous regression with categorical risk classification (e.g., Dropout Risk probability estimates).
4. **Automated Mentoring Notifications:** Implement automated email alerts alerting faculty advisors when student trajectories dip below critical thresholds.

REFERENCES & ACADEMIC BIBLIOGRAPHY

1. Romero, C., & Ventura, S. (2010). Educational data mining: a review of the state of the art. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 40(6), 601-618.
2. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Science & Business Media.
3. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
4. Baker, R. S., & Inventado, P. S. (2014). Educational data mining and learning analytics. In *Learning Analytics* (pp. 61-75). Springer, New York, NY.
5. Cortez, P., & Silva, A. M. G. (2008). Using data mining to predict secondary school student performance. In *Proceedings of 5th Annual Future Business Technology Conference* (pp. 5-12).
6. McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference* (Vol. 445, pp. 51-56).
7. Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media, Inc.
8. Banks, F., & Porcello, E. (2020). *Learning React: Modern Patterns for Developing React Apps*. O'Reilly Media, Inc.
9. Wold, S., Sjöström, M., & Eriksson, L. (2001). PLS-regression: a basic tool of chemometrics. *Chemometrics and Intelligent Laboratory Systems*, 58(2), 109-130.
10. Scikit-learn Developers. (2024). *Linear Models: Ordinary Least Squares Documentation*. https://scikit-learn.org/stable/modules/linear_model.html

APPENDIX: CORE SOURCE CODE LISTINGS

Appendix A: Model Training & Evaluation Script (train_model.py)

```
# train_model.py - Multiple Linear Regression Training Script
import os, joblib
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

# 1. Load Dataset
df = pd.read_csv('data/student_performance.csv')
features = ['study_hours', 'attendance', 'previous_score', 'assignment_score', 'sleep_hours']
X = df[features]
y = df['final_score']

# 2. 80/20 Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Fit OLS Linear Regression
model = LinearRegression()
model.fit(X_train, y_train)

# 4. Evaluate Metrics
y_pred = model.predict(X_test)
mae = mean_absolute_error(y_test, y_pred) # 1.86 marks
r2 = r2_score(y_test, y_pred) # 0.9685 (96.8% variance)

# 5. Save Serialized Artifact
joblib.dump(model, 'models/student_performance_model.pkl')
```

Appendix B: Flask Defensive Validation Core (backend/app.py)

```
# backend/app.py - Defensive Input Validation Pipeline
for canonical_name, spec in FIELD_VALIDATION_RULES.items():
    val = raw_data.get(canonical_name)
    if val is None or isinstance(val, bool) or val == '':
        return jsonify({'error': f"{spec['label']} must be a valid number."}), 400
    try:
        numeric_val = float(val)
    except (ValueError, TypeError):
        return jsonify({'error': f"{spec['label']} must be a valid number."}), 400
    if math.isnan(numeric_val) or math.isinf(numeric_val):
        return jsonify({'error': f"{spec['label']} cannot be NaN or Infinite."}), 400
    if not (spec['min'] <= numeric_val <= spec['max']):
        return jsonify({'error': f"{spec['label']} must be between {spec['min']} and {spec['max']}."}), 400
    validated_features[canonical_name] = numeric_val

# Inference & Category Mapping
raw_pred = float(model.predict(pd.DataFrame([validated_features]))[0])
predicted_score = int(round(max(0.0, min(100.0, raw_pred))))
```